

FLUXOGRAMAS — SQUAD 16

Versão 2.0 — CRUD Usuários e Produtos (corrigidos) + 5 novos fluxogramas

Este documento contém todos os fluxogramas do sistema. Os dois fluxogramas existentes (CRUD Usuários e CRUD Produtos) foram revisados e corrigidos. Os 5 novos fluxogramas cobrem os fluxos críticos ausentes na v1.

Fluxo 1 — CRUD Usuários [CORRIGIDO]

■ v1: Autorização verificava 'gerente de serviços/operacional/geral' mas User não tinha campo role — nenhum dado no banco suportava esse check. DELETE não verificava se usuário era líder de squad.

✓ v2: Autorização usa User.role (GAME_MASTER). DELETE valida User.leader antes de prosseguir.

#	Ator	Passo	Saída / Dado	Condição / Desvio
1	Sistema	Receber requisição HTTPS	Request com JWT	—
2	Middleware auth	Validar JWT	Payload decodificado (userId, role)	JWT inválido → 401 Unauthorized
3	Middleware role	Verificar se role == GAME_MASTER	role confirmado	role != GAME_MASTER → 403 Forbidden
4	Router	Identificar método HTTP	Método (GET/POST/PUT/DELETE)	
—	—	■■■ Caminho GET ■■■	—	—
5a	Controller	Buscar todos os usuários no banco	Lista de usuários	—
6a	Controller	Retornar lista	200 OK + JSON	—
—	—	■■■ Caminho POST ■■■	—	—
5b	Controller	Validar corpo da requisição (name, email, password, role, squadId?)	Dados validados	Dados inválidos → 400 Bad Request
6b	Controller	Criar usuário no banco (Prisma)	Novo User criado	Email duplicado → 409 Conflict
7b	Controller	Retornar usuário criado	201 Created + JSON	—
—	—	■■■ Caminho PUT ■■■	—	—
5c	Controller	Verificar se usuário existe (findById)	User encontrado	Não encontrado → 404 Not Found
6c	Controller	Validar dados de atualização	Dados validados	Dados inválidos → 400 Bad Request
7c	Controller	Atualizar usuário no banco	User atualizado	—
8c	Controller	Retornar usuário atualizado	200 OK + JSON	—
—	—	■■■ Caminho DELETE ■■■	—	—
5d	Controller	Verificar se usuário existe (findById)	User encontrado	Não encontrado → 404 Not Found
6d	Controller	Verificar se User.leader == true	Resultado da verificação	leader == true → 409 Conflict: 'Transfira a liderança antes de remover'
7d	Controller	Deletar usuário do banco	User removido	—
8d	Controller	Retornar confirmação	200 OK + { deleted: true }	—

Fluxo 2 — CRUD Produtos [CORRIGIDO]

■ v1: Autorização verificava 'gerente de abastecimento ou gerente geral' sem campo role no banco. DELETE não considerava produtos referenciados em Inventory ou RoundConfigItem.

✓ v2: Autorização usa User.role (GAME_MASTER). DELETE verifica integridade referencial antes de excluir.

#	Ator	Passo	Saída / Dado	Condição / Desvio
1	Sistema	Receber requisição HTTPS	Request com JWT	—
2	Middleware auth	Validar JWT	Payload decodificado	JWT inválido → 401 Unauthorized
3	Middleware role	Verificar se role == GAME_MASTER	role confirmado	role != GAME_MASTER → 403 Forbidden
4	Router	Identificar método HTTP	Método (GET/POST/PUT/DELETE)	
—	—	■■■ Caminho GET ■■■	—	—
5a	Controller	Buscar todos os produtos no banco	Lista de produtos	—
6a	Controller	Retornar lista	200 OK + JSON	—
—	—	■■■ Caminho POST ■■■	—	—
5b	Controller	Validar corpo (name, purchasePrice, salePrice)	Dados validados	Dados inválidos → 400 Bad Request
6b	Controller	Criar produto no banco	Novo Product criado	Nome duplicado → 409 Conflict
7b	Controller	Retornar produto criado	201 Created + JSON	—
—	—	■■■ Caminho PUT ■■■	—	—
5c	Controller	Verificar se produto existe	Product encontrado	Não encontrado → 404 Not Found
6c	Controller	Validar dados de atualização	Dados validados	Dados inválidos → 400 Bad Request
7c	Controller	Atualizar produto no banco	Product atualizado	—
8c	Controller	Retornar produto atualizado	200 OK + JSON	—
—	—	■■■ Caminho DELETE ■■■	—	—
5d	Controller	Verificar se produto existe	Product encontrado	Não encontrado → 404 Not Found
6d	Controller	Verificar integridade referencial: existe Inventory ou RoundConfigItem com productId?	Resultado da verificação	Referências existem → 409 Conflict 'Produto está em uso' — remova as referências
7d	Controller	Deletar produto do banco	Product removido	—
8d	Controller	Retornar confirmação	200 OK + { deleted: true }	—

Fluxo 3 — Autenticação e Controle de Acesso [NOVO]

Fluxo base que precede qualquer operação protegida no sistema. Implementado como middlewares reutilizáveis: `authMiddleware.js` e `roleMiddleware.js`.

#	Ator	Passo	Saída / Dado	Condição / Desvio
1	Cliente (React)	Enviar credenciais via POST <code>/auth/login</code> { email, password }	Corpo da requisição	—
2	Controller	Buscar usuário por email no banco	User ou null	Não encontrado → 401 Unauthorized (mensagem genérica — não revela
3	Controller	Comparar password com hash (<code>bcrypt.compare</code>)	Boolean (match)	Senha errada → 401 Unauthorized
4	Controller	Gerar JWT assinado { <code>userId</code> , <code>role</code> , <code>squadId</code> , <code>exp</code> : 8h }	Token JWT	—
5	Controller	Retornar token ao cliente	200 OK + { token, user: { <code>id</code> , <code>name</code> , <code>role</code> } }	—
—	—	■■ Fluxo de uso do token (requisições subsequentes) ■■	—	—
6	<code>authMiddleware</code>	Extrair token do header <code>Authorization: Bearer <token></code>	Token extraído	Header ausente → 401 Unauthorized
7	<code>authMiddleware</code>	Verificar assinatura e expiração do JWT	Payload decodificado	Token inválido/expirado → 401 Unauthorized
8	<code>authMiddleware</code>	Injetar <code>req.user = { id, role, squadId }</code>	<code>req.user</code> disponível	—
9	<code>roleMiddleware</code>	Verificar se <code>req.user.role</code> está na lista de roles permitidas	Acesso permitido	Role insuficiente → 403 Forbidden
10	Router	Encaminhar para o controller da rota	Processamento da requisição	—
—	—	■■ Redirecionamento por role (após login) ■■	—	—
11a	Frontend	Se <code>role == GAME_MASTER</code> → redirecionar para <code>/admin/dashboard</code>	Dashboard do GM	—
11b	Frontend	Se <code>role == PLAYER</code> → redirecionar para <code>/store/dashboard</code>	Dashboard da loja	—
11c	Frontend	Se <code>role == OBSERVER</code> → redirecionar para <code>/ranking</code>	Tela de ranking (somente leitura)	—

Fluxo 4 — Configuração de Rodada (RoundConfig) [NOVO]

Fluxo pelo qual um PLAYER submete as decisões estratégicas da rodada: preço de venda, volume por produto e despesas. Inclui validação de estoque e modo preview (simular sem salvar).

#	Ator	Passo	Saída / Dado	Condição / Desvio
1	PLAYER	Acessar tela de configuração da rodada ativa	Round com status OPEN	Round não OPEN → exibir mensa
2	Sistema	Carregar produtos disponíveis + estoque da loja (Product + Inventory por storeId)	Lista de produtos com quantity disponível	
3	PLAYER	Preencher para cada produto: salePrice, salesVolume	RoundConfigItem por produto	
4	PLAYER	Preencher despesas da loja: fixedExpenses, variableExpenses	RoundConfig header	—
5	Sistema (frontend)	Validar em tempo real: - salesVolume <= Inventory.quantity - salePrice > 0 - salePrice >= purchasePrice (aviso, não bloqueio)	Feedback visual por campo	Campo inválido → exibir aviso inlin
6	PLAYER	Clicar em 'Simular' (preview)	Chamada: POST /simulation/preview	
7	SimulationService	Executar calcularDRE() com dados do formulário SEM persistir no banco	DRE calculado em memória	—
8	Sistema	Exibir resultado do preview na tela (área marcada: 'Simulação — não confirmada')	DRE exibido visualmente	—
9	PLAYER	Clicar em 'Confirmar estratégia'	Chamada: POST /rounds/{id}/config	
10	Controller	Validar se já existe RoundConfig para (roundId, storeId)	Verificação de unicidade	Já existe → 409 Conflict: 'Você já s
11	Controller	Validar todos os campos server-side (não confiar apenas no frontend)	Dados validados	Dados inválidos → 400 Bad Reque
12	Prisma	Persistir RoundConfig + RoundConfigItems em transação única	RoundConfig salvo com itens	Erro de banco → 500 Internal Serv
13	Sistema	Retornar confirmação ao cliente	201 Created + { roundConfigId }	

Fluxo 5 — Motor de Simulação e Cálculo do DRE [NOVO]

Fluxo disparado pelo GAME_MASTER ao encerrar uma rodada. O sistema processa todas as RoundConfigs submetidas, calcula o DRE de cada loja e persiste os FinancialResults. É o coração do sistema.

#	Ator	Passo	Saída / Dado	Condição / Desvio
1	GAME_MASTER	Clicar em 'Encerrar Rodada' PATCH /rounds/:id/close	Requisição de encerramento	Role != GAME_MASTER → 403 Forbidden
2	RoundController	Verificar se Round.status == OPEN	Status confirmado	Status != OPEN → 409 Conflict
3	RoundController	Atualizar Round.status = PROCESSING	Round em processamento	—
4	SimulationService	Buscar todas as RoundConfigs da rodada (include: items, store, items.product)	Lista de RoundConfigs	Nenhuma config → encerrar rodada
5	SimulationService	Para cada RoundConfig, buscar Inventory da store	Inventory por storeId	—
6	SimulationService	Para cada RoundConfigItem: effectiveVolume = min(item.salesVolume, inventory.quantity)	Volume efetivo por produto	Se inventory = 0 para o produto → 0
7	financeService calcularDRE()	Calcular por item: • itemRevenue = salePrice * effectiveVolume • itemCost = purchasePrice * effectiveVolume	Subtotais por produto	—
8	financeService calcularDRE()	Agregar totais da loja: • grossRevenue = sum(itemRevenue) • costs = sum(itemCost) • expenses = fixedExpenses + variableExpenses	Totais agregados	—
9	financeService calcularDRE()	Calcular resultados finais: • grossProfit = grossRevenue - costs • netProfit = grossProfit - expenses • netMargin = grossRevenue != 0 ? (netProfit/grossRevenue)*100 : 0	DRE completo	grossRevenue = 0 → netMargin = 0
10	SimulationService	Persistir FinancialResult no banco (vinculado a roundId, storeId, roundConfigId)	FinancialResult salvo	Erro → rollback, manter Round.status
11	SimulationService	Atualizar Inventory: descontar effectiveVolume de cada produto	Inventory atualizado	—
12	RoundController	Atualizar Round.status = CLOSED	Rodada encerrada	—
13	Sistema	Notificar players: rodada encerrada, resultados disponíveis	Notificação (polling ou WebSocket)	—

Fluxo 6 — Gestão de Rodadas pelo Game Master [NOVO]

Fluxo completo do ciclo de vida de uma rodada: criação, abertura para configurações, monitoramento e encerramento com disparo do motor de simulação.

#	Ator	Passo	Saída / Dado	Condição / Desvio
1	GAME_MASTER	Acessar painel de gerenciamento de rodadas	Dashboard do GM	—
2	GAME_MASTER	Clicar em 'Nova Rodada' POST /rounds { number, startDate, endDate }	Requisição de criação	Já existe rodada com status OPEN
3	RoundController	Criar Round com status OPEN	Round criado (status: OPEN)	—
4	Sistema	Notificar todos os PLAYERS: nova rodada disponível	Notificação enviada	—
5	GAME_MASTER	Monitorar submissões: GET /rounds/:id/configs (lista quais squads já submeteram)	Lista de RoundConfigs submetidas	—
6	GAME_MASTER	(Opcional) Alterar parâmetros globais enquanto rodada OPEN PUT /products/:id { purchasePrice }	Produto atualizado	Alteração afeta apenas RoundConfigs
7	GAME_MASTER	(Opcional) Criar evento aleatório POST /events { type, modifier, roundId }	GameEvent persistido	Evento visível para PLAYERS antes da rodada
8	GAME_MASTER	Clicar em 'Encerrar Rodada' PATCH /rounds/:id/close	Disparo do motor de simulação	Ver Fluxo 5 — Motor de Simulação
9	Sistema	Processar todos os DREs (Fluxo 5)	FinancialResults persistidos	—
10	Sistema	Atualizar Round.status = CLOSED	Rodada encerrada	—
11	GAME_MASTER	Visualizar ranking e resultados GET /ranking?roundId=X	Ranking calculado on-demand	—
12	GAME_MASTER	Iniciar próxima rodada (volta ao passo 2)	Novo ciclo	—

Fluxo 7 — Ranking e Visualização de Resultados [NOVO]

Fluxo de acesso ao ranking e ao DRE individual após o encerramento de uma rodada. O ranking é calculado on-demand — não existe campo score estático na entidade Squad.

#	Ator	Passo	Saída / Dado	Condição / Desvio
1	PLAYER / GAME_MASTER	Acessar tela de resultados da rodada GET /rounds/:id/results	Requisição autenticada	Round.status != CLOSED → 404 o
2	ResultsController	Buscar FinancialResult do usuário (filtrado por storeId da loja do squad)	FinancialResult da loja	Loja sem FinancialResult → exibir
3	Sistema	Montar DRE individual: grossRevenue, costs, expenses, grossProfit, netProfit, netMargin	DRE formatado	—
4	Sistema	Exibir DRE com formatação: • valores monetários em R\$ • netMargin em % • netProfit em verde (>0) ou vermelho (<0)	DRE renderizado	netMargin com grossRevenue=0 —
5	PLAYER / GAME_MASTER	Acessar ranking geral GET /ranking?roundId=X	Requisição autenticada	—
6	RankingService	Buscar todos os FinancialResults da rodada include: store.squad	Lista de resultados	—
7	RankingService	Ordenar por netMargin DESC (critério primário) e netProfit DESC (de empate)	Lista ordenada	—
8	RankingService	Mapear resultado: { position, squadName, storeName, netMargin, netProfit, grossRevenue }	Ranking formatado	—
9	Sistema	Retornar ranking ao cliente	200 OK + JSON array	—
10	Sistema	Exibir ranking com posição, nome do squad e métricas (NÃO expor detalhes individuais do DRE dos concorrentes)	Ranking renderizado	—
11	PLAYER	(Perfil profissional) Comparar próprio DRE com médias do ranking exibidas na mesma tela	Análise contextualizada	—